

HTWG Konstanz

**Ein lebender Wissensgraph für Coding
Agents:
Kontexttransparenz und strukturelle
Context Assembly
in der agentischen Softwareentwicklung**

Exposé zur Masterarbeit

Betreuer-Ausschreibung: „*Mehr Effizienz mit Coding Agents? Kontexttransparenz und
Kontext-Engineering*“

Joschka Peeters

Matrikelnummer: 301750

11. Mai 2026

Betreuer: Prof. Dr. Rainer Müller

Inhaltsverzeichnis

1	Problemstellung und Motivation	3
2	Stand der Forschung	4
3	Forschungsfragen und Hypothesen	8
4	Das Artefakt: Living Knowledge Graph	9
5	Methodisches Vorgehen	13
6	Vorläufige Gliederung der Arbeit	15
7	Zeitplan	16
8	Einarbeitungshinweise	19
	Literaturverzeichnis	23

1 Problemstellung und Motivation

Coding Agents sind in der professionellen Softwareentwicklung angekommen. Coding Agents wie SWE-agent übernehmen nicht länger nur Autovervollständigung, sondern planen Aufgaben, rufen Werkzeuge auf, lesen Dateien und koordinieren Änderungen über mehrere Module hinweg (J. Yang u. a. 2024). Kommerzielle Werkzeuge wie Claude Code, Cursor und GitHub Copilot Workspace setzen dieses Paradigma mittlerweile im Produktiveinsatz um. An der Literatur lässt sich ein messbarer Produktivitätsgewinn belegen: Entwicklerinnen und Entwickler schließen abgegrenzte Aufgaben mit AI-Unterstützung bis zu 55,8 % schneller ab als ohne (Peng u. a. 2023). Der SWE-bench-Benchmark ist ein Datensatz realer GitHub-Issues, der als standardisiertes Testwerkzeug für KI-Agenten eingesetzt wird (Jimenez u. a. 2024). Er zeigt, dass selbst die stärksten Modelle reale Repository-Aufgaben nur dann zuverlässig lösen, wenn sie Multi-File-Zusammenhänge verstehen und in langen Kontexten navigieren können.

Ein wesentliches Hindernis liegt dabei nicht allein im Modell, sondern in der *Kontextbereitstellung*: Was ein Coding Agent über eine Codebase weiß (welche Dateien er gelesen hat, welche Abhängigkeiten er kennt, welche Konventionen ihm vertraut sind) bestimmt entscheidend, ob er eine Aufgabe korrekt löst oder in produktionsfremden Mustern halluziniert (Li u. a. 2026).

Das Reaktivitätsproblem moderner Coding Agents

Aktuelle Coding Agents explorieren Codebases *reaktiv*. Das bedeutet, dass der Agent erst dann Dateien öffnet und nach relevanten Funktionen sucht, wenn er eine konkrete Aufgabe erhält. Diese Strategie ist teuer und fehleranfällig. Li u. a. (2026) dokumentieren, dass Coding Agents systematisch *mehr* Kontext abrufen als sie tatsächlich nutzen, und dabei vorher besuchte Dateien wiederholt aufsuchen (ein Muster redundanter Exploration). Suri u. a. (2026) zeigen, dass Agents bei unklaren Aufgabenbeschreibungen in eine ziellose Überexploration geraten. Ein schlecht formulierter Bug-Report führt in ihren Experimenten zu 21 Suchschritten ohne Ergebnis. Mit einer strukturierten Aufgabenbeschreibung löst derselbe Agent das gleiche Problem in sechs Schritten.

Es gibt bereits proaktive Ansätze, die Kontext vor der Agentenausführung aufbereiten. Xia u. a. (2024) lokalisiert Fehlerstellen in einem vorgelagerten Dreiphasen-Prozess. Suri u. a. (2026) reichert die Aufgabenbeschreibung durch eine Vorab-Exploration des Repositories an. Beide Systeme analysieren die Codebase jedoch aufgabenbezogen und zum Zeitpunkt der jeweiligen Anfrage. Eine fortlaufende Synchronisation mit einer aktiv weiterentwickelten Codebase ist nicht vorgesehen.

Das Transparenzproblem

Mindestens ebenso schwerwiegend wie das Reaktivitätsproblem ist das *Transparenzproblem*: Wenn ein Coding Agent eine falsche Lösung produziert, kann der Entwickler nicht nachvollziehen, welcher Kontext zu welcher Entscheidung geführt hat. Kontext ist eine Black Box. Fehlt einer Funktion die Kenntnis ihrer Aufrufer? Hat der Agent das zugehörige Schema nicht gelesen? Ohne Antwort auf diese Fragen bleibt die Fehlersuche schwer nachvollziehbar. Erste Ansätze zur Agenten-Transparenz, etwa AgentStepper (Hutter und Pradel 2026), helfen Entwicklern, Ausführungstrajektorien nachzuvollziehen. Die vorgelagerte Inspizierbarkeit der Kontextauswahl (*Retrieval-Transparenz*), also welche Quellen warum in den Kontext aufgenommen wurden, findet dabei bislang kaum Beachtung.

Zielsetzung

Ziel dieser Arbeit ist die Entwicklung und Evaluation eines *Living Knowledge Graph* als Kontextinfrastruktur für Coding Agents. Das System repräsentiert eine Codebase als strukturierten, automatisch synchronisierten Graphen aus Dateien, Funktionen, Abhängigkeiten und Testergebnissen. Erhält das System eine Aufgabe, sucht es selbstständig die passenden Code-Abschnitte heraus und gibt sie dem Agenten weiter. Es erklärt dabei, warum jede Stelle ausgewählt wurde. Das System lässt sich direkt in bestehende Werkzeuge wie Claude Code und Cursor einbinden.

Diese Arbeit zeigt, wie Coding Agents weniger Zeit damit verbringen müssen, sich in einer Codebase zu orientieren. Außerdem sollen Entwickler besser nachvollziehen können, warum ein Agent eine bestimmte Lösung vorschlägt.

2 Stand der Forschung

Agentische Ansätze für Software Engineering

SWE-bench ist ein Benchmark aus realen GitHub-Issues (Jimenez u. a. 2024), der misst, wie viele davon ein Agent korrekt löst. Die Aufgaben erfordern Änderungen in mehreren Dateien, komplexes Reasoning und Interaktion mit Ausführungsumgebungen. Klassische Sprachmodelle ohne Werkzeuge schaffen das nicht.

J. Yang u. a. (2024) zeigen mit SWE-agent, dass eine spezialisierte *Agent-Computer Interface* (ACI, eine strukturierte Schnittstelle zwischen Modell und Dateisystem) die Lösungsrate auf SWE-bench von 3,8 % (bisherige RAG-Baseline) auf 12,5 % anhebt.

SWE-agent steht für den *reaktiven* Ansatz: Der Agent entscheidet autonom, welche Dateien er liest, welche Suchen er ausführt und welche Schritte er unternimmt.

Einen gegensätzlichen Ansatz verfolgt Xia u. a. (2024) mit Agentless. Statt autonomer Exploration verwendet ein dreiphasiger, extern verwalteter Prozess (Lokalisierung, Patchgenerierung, Testvalidierung) strukturierte Kontextzusammenstellung. Auf SWE-bench Lite (einer 300-Aufgaben-Teilmenge des vollen SWE-bench) erreicht Agentless 32 % bei nur 0,70 USD pro Fix. Das zeigt, dass prozedurale Kontextvorbereitung autonome Agentenexploration schlagen kann. Das gelingt zu einem Bruchteil der Kosten.

Wissensgraph-basierte Ansätze

Ouyang u. a. (2025) stellen 2025 RepoGraph vor. Es ist ein Plug-in-Modul zu bestehenden Software-Engineering-Agenten. Es analysiert den Quellcode via AST (Abstract Syntax Tree, ein automatisch erstellter Syntaxbaum) und baut daraus einen Repository-Graphen aus Referenz- und Definitionsknoten mit Aufruf- und Enthaltensrelationen. Als Ergänzungsmodul erzielt RepoGraph auf SWE-bench eine durchschnittliche relative Verbesserung von 32,8 % gegenüber denselben Agenten ohne RepoGraph (RAG, Agentless, SWE-agent), ohne die Kernsysteme zu verändern.

B. Yang u. a. (2025) gehen mit KGCompass weiter. Ein repository-bewusster Wissensgraph verknüpft Issues, Pull Requests, Dateien, Klassen und Funktionen. Ein pfadgeführter Reparaturmechanismus nutzt Mehr-Hop-Traversierungen, um relevante Codestellen zu identifizieren. KGCompass erreicht 58,3 % auf SWE-bench Lite und eine Fehlerlokalisierungsgenauigkeit von 56,0 %. Besonders aufschlussreich: 89,7 % der korrekt reparierten Patches erfordern Mehr-Hop-Verbindungen im Graphen. Das sind Codestellen, die reine LLMs ohne Graphtraversierung systematisch übersehen.

Proaktive versus reaktive Kontextbereitstellung

Die Spannung zwischen proaktiver Vorbereitung und reaktiver Exploration ist ein eigenständiges Forschungsthema. Suri u. a. (2026) zeigen mit CodeScout, dass proaktive Kontextanreicherung vor der Agentenausführung der reaktiven Eigenexploration überlegen ist. Letztere neigt zu Über-Exploration und repetitiven Fix-Mustern, während CodeScout durch strukturierte Vorab-Exploration präziseren Kontext liefert.

Ergänzend dokumentieren Li u. a. (2026) mit ContextBench auf 1 136 Issue-Aufgaben aus 66 Repositories in acht Programmiersprachen: Aktuelle Coding Agents priorisieren systematisch Recall über Präzision (es wird mehr abgerufen als nötig), und es bestehen erhebliche Lücken zwischen dem explorierten und dem tatsächlich genutzten

Kontext. Komplexere Agent-Architekturen verbessern die Kontextabrufqualität dabei nur marginal.

Das von Liu u. a. (2024) dokumentierte *Lost-in-the-Middle*-Phänomen gibt diesem Befund eine modellseitige Erklärung: Selbst als Long-Context-tauglich beworbene Modelle nutzen Informationen in der Mitte langer Eingaben deutlich schlechter als an Anfang und Ende. Proaktive, strukturell geordnete Kontextzusammenstellung ist daher nicht nur eine Effizienzfrage, sondern eine Voraussetzung für zuverlässige Modellnutzung.

Statische Kontext-Dateien als Agenten-Baseline

Eine weitere Herangehensweise untersucht den Wert manuell gepflegter Kontext-Dateien (CLAUDE.md, AGENTS.md, copilot-instructions.md) für Coding Agents. Gloaguen u. a. (2026) zeigen in einer systematischen Studie auf SWE-bench (AGENTbench, 138 Aufgaben aus 12 Repositories), dass vom Entwickler verfasste AGENTS.md-Dateien die Lösungsrate um durchschnittlich 4 % verbessern. LLM-generierte Dateien dagegen senken sie um 3 %. Beide Varianten erhöhen die Inferenzkosten um mehr als 20 %. Lulla u. a. (2026) ergänzen auf Basis von 124 Pull Requests aus 10 Repositories, dass AGENTS.md-Dateien die mediane Bearbeitungszeit um 28,6 % und den Output-Token-Verbrauch um 16,6 % reduzieren (der Agent braucht mit besserem Kontext weniger Schritte).

Diese Befunde belegen: Statische Entwickler-Dokumentation hat einen messbaren, aber begrenzten Effekt auf Lösungsrate und Effizienz. Ein direkter Vergleich mit graphbasierter Kontextzusammenstellung fehlt in der Literatur bislang.

Transparenz von Coding Agents

Außerdem interessant ist die Frage nach der Inspizierbarkeit agentischer Prozesse. Hutter und Pradel (2026) stellen AgentStepper vor, einen interaktiven Debugger für Coding Agents (SWE-agent, ExecutionAgent, RepairAgent), der Entwicklern ermöglicht, Ausführungs-Trajektorien (die Abfolge aller Schritte, die ein Agent bei einer Aufgabe durchführt) schrittweise zu durchsuchen, Prompt-Verläufe einzusehen und Agenten-Entscheidungen nachzuverfolgen. In einer kontrollierten Studie mit 12 Teilnehmenden stieg die Bug-Erkennungsrate von 17 % auf 60 %. Diese Arbeit belegt, dass Einblick in die Ausführungsschritte die Fähigkeit von Entwicklern zur Fehlerkorrektur messbar verbessert.

Was AgentStepper und verwandte Ansätze nicht adressieren, ist die vorgelagerte Frage: Welche Codestellen hat das System ausgewählt und warum? Diese Frage ist bislang

nicht als eigenständige Forschungsfrage untersucht.

Integrationsinfrastruktur: MCP

Im November 2024 hat Anthropic das *Model Context Protocol* (MCP) als offenen Standard für die Integration von LLM-Anwendungen mit externen Werkzeugen und Datenquellen veröffentlicht (Anthropic 2024). MCP definiert ein JSON-RPC-basiertes Kommunikationsprotokoll zwischen Client (Coding Agent) und Server (Datenprovider) mit den Primitiven Prompts, Resources und Tools. Für diese Arbeit ist MCP die natürliche Integrationsschicht: Ein MCP-Server kann den Wissensgraphen direkt als aufrufbares Tool in Claude Code, Cursor und kompatible Clients einbetten.

Was diese Arbeit untersucht

Die dargestellten Arbeiten zeigen drei Dinge. Graphbasierte Kontextzusammenstellung verbessert die Agent-Performance messbar. Statische Entwickler-Dokumentation hat einen begrenzten positiven Effekt auf Lösungsrate und Effizienz. Transparenz über Ausführungsschritte hilft Entwicklern bei der Fehlerkorrektur. Daraus ergeben sich drei Fragen, die bislang nicht systematisch untersucht wurden.

Erstens: Wie lässt sich ein Wissensgraph mit einer aktiv weiterentwickelten Codebase synchron halten? Die bisherigen graphbasierten Systeme (RepoGraph, KGCompass) analysieren das Repository zum Zeitpunkt einer Aufgabe. Eine kontinuierliche Aktualisierung adressiert keines dieser Systeme.

Zweitens: Welche Codestellen hat das System ausgewählt und warum? Die bisher untersuchte Transparenz beschränkt sich auf die Ausführungsschritte nach Agentstart. Welche Kontextentscheidungen davor getroffen wurden, ist bislang nicht als eigenständige Frage untersucht.

Drittens: Wie schneidet graph-basierter Kontext im direkten Vergleich mit statischer Dokumentation und reaktiver Exploration ab? Gloaguen u. a. (2026) vergleichen nur statische Kontext-Dateien mit einer Baseline ohne Kontext-Dateien. Ein solcher Drei-Wege-Vergleich steht aus.

Die vorliegende Arbeit untersucht diese drei Fragen mit einem *Living Knowledge Graph*, der inkrementell mit der Codebase synchronisiert wird, die Kontextauswahl inspizierbar macht und als MCP-Server in bestehende Coding-Agent-Umgebungen integrierbar ist.

3 Forschungsfragen und Hypothesen

Forschungsfragen

- HF1** Verbessert proaktiv graph-assemblierter Kontext die Effizienz von Coding Agents (gemessen an Token-Verbrauch, Iterationsanzahl und Erstkorrektheit, d. h. Rechenkosten, Agentenschritten und Erstlösungsqualität) gegenüber reaktiver Agent-Eigenexploration (B1) und gegenüber statischer Entwickler-Dokumentation (B2)?
- HF2** Welche Graph-Traversal-Strategien (strukturell-deklarativ, historisch, testbasiert) liefern für welche Aufgabentypen (isolierte Bugfixes, Multi-File-Refactorings, neue Feature-Implementierungen) den präzisesten Kontext?
- HF3** Erhöht die transparente Visualisierung der Kontextauswahl die Fähigkeit von Entwicklern, Fehler im Agent-Output zu erkennen und gezielt zu korrigieren?

Hypothesen

- H1** Graph-basiert proaktiv assemblierter Kontext reduziert Token-Verbrauch und Iterationsanzahl gegenüber reaktiver Exploration und gegenüber statischer Entwickler-Dokumentation. Diese Hypothese stützt sich auf drei komplementäre Befunde: Li u. a. (2026) dokumentieren, dass reaktive Agents erheblich mehr Kontext abrufen als sie nutzen und vorher besuchte Dateien wiederholt aufsuchen. Xia u. a. (2024) zeigen, dass prozedurale Kontextvorbereitung reaktive Agentenexploration auf SWE-bench Lite mit 32 % und nur 0,70 USD pro Fix übertrifft. Gloaguen u. a. (2026) zeigen, dass statische Kontext-Dateien zwar die Lösungsrate um 4 % verbessern, die Inferenzkosten aber um mehr als 20 % erhöhen, was nahelegt, dass aufgabenspezifisch gefilterte Kontextzusammenstellung effizienter sein kann. Ein strukturell aufgebauter Graph, der nur den direkt abhängigen Teilgraphen ausliefert, sollte dieses Effizienzpotenzial ausschöpfen.
- H2** Multi-Hop-Graphtraversierung über Aufruf- und Abhängigkeitskanten liefert für Multi-File-Aufgaben präziseren Kontext als flaches Dateisuchen. Diese Hypothese basiert auf dem zentralen KGCompass-Befund: 89,7 % der korrekt reparierten Patches erfordern Mehr-Hop-Verbindungen im Graphen und betreffen Codestellen, die reine LLMs ohne Graphtraversierung systematisch übersehen (B. Yang u. a. 2025). Der relative Leistungsgewinn von RepoGraph über bestehende Systeme (+32,8 %) bestätigt den Mehrwert struktureller Repräsentation (Ouyang u. a. 2025).
- H3** Entwickler, denen die *Retrieval-Transparenz* (also welche Graph-Knoten warum

in den Kontext aufgenommen wurden) als inspizierbarer Entscheidungspfad präsentiert wird, erkennen mehr Agent-Fehler und benötigen weniger Korrekturiterationen als Entwickler ohne diese Information. Die Hypothese ist explorativ. Verwandte Arbeiten zeigen, dass Transparenz über Ausführungs-Trajektorien die Fehlererkennungsrate messbar verbessert (Hutter und Pradel 2026); die vorgelagerte Frage der Retrieval-Transparenz ist bislang empirisch unerforscht. Ergänzend zeigt Liu u. a. (2024), dass fehlerhafte Kontextpositionierung zu Modellfehlern führt, was nahelegt, dass sichtbarer Kontext Entwicklern gezielteren Eingriff ermöglicht.

4 Das Artefakt: Living Knowledge Graph

Das zentrale Artefakt dieser Arbeit ist ein System, das eine Codebase als strukturierten Wissensgraphen repräsentiert und diesen als proaktive Kontextinfrastruktur für Coding Agents bereitstellt. *Living* bezeichnet dabei die Eigenschaft, dass der Graph nach jedem Commit automatisch aktualisiert wird und damit dauerhaft den aktuellen Codestand abbildet (siehe Schicht 2). Das System gliedert sich in drei Schichten; Abbildung 1 gibt einen Überblick.

Schicht 1: Ingestion und Graph-Aufbau

Ein auf *tree-sitter* basierender Parser verarbeitet Python-, JavaScript- und TypeScript-Quellcode und extrahiert daraus einen strukturierten Graphen in Neo4j:

Knotentyp	Bedeutung
File	Quellcode-Datei mit Pfad und Sprache
Function	Funktion oder Methode mit Signatur und Zeilenbereich
Class	Klasse oder Interface
TestCase	Testfunktion mit Referenz zum getesteten Symbol
Schema	Daten- oder API-Schema (z. B. Sanity- oder Pydantic-Schema)

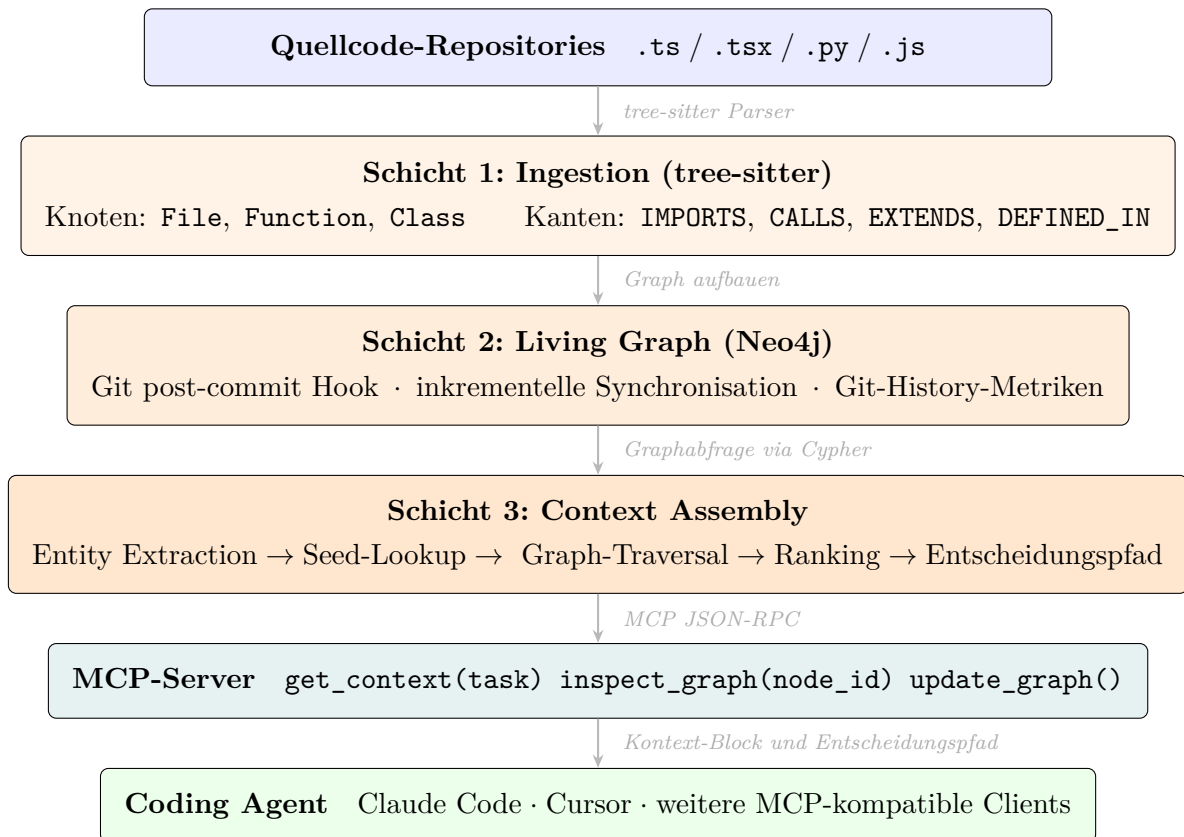


Abbildung 1: Systemarchitektur des *Living Knowledge Graph*. Der Coding Agent erhält über den MCP-Server proaktiv assemblierten, aufgabenspezifischen Kontext, ohne die Codebase reaktiv zu explorieren.

Kantentyp	Bedeutung
IMPORTS	Datei importiert ein Symbol aus einer anderen Datei
CALLS	Funktion ruft eine andere Funktion auf
EXTENDS	Klasse erbt von einer anderen Klasse
DEFINED_IN	Funktion oder Klasse ist in einer bestimmten Datei definiert
TESTS	Testfunktion testet ein bestimmtes Symbol
DOCUMENTS	Docstring oder Kommentar dokumentiert ein Symbol

Ergänzend werden Git-History-Metriken als Knotenattribute gespeichert: Änderungshäufigkeit und letzte Commit-Nachricht. Diese ermöglichen risikogewichtetes Ranking: Häufig modifizierte Dateien werden bei gleicher Hop-Distanz bevorzugt.

Schicht 2: Living Graph mit inkrementeller Git-Synchronisation

Ein Git post-commit Hook analysiert nach jedem Commit, welche Dateien sich verändert haben. Nur die betroffenen Knoten und ihre direkten Kanten werden neu geparkt und aktualisiert; ein vollständiger Rebuild entfällt. Damit bleibt der Graph dauerhaft synchron mit dem Codestand, ohne dass manuelle Wartung nötig ist. Dies ist die *Living*-Eigenschaft des Systems: Der Wissensgraph ist kein statisches Dokument, sondern ein kontinuierlich aktualisiertes Abbild der Codebase.

Schicht 3: Proaktive Context Assembly

Für eine eingehende Aufgabenbeschreibung läuft folgende Pipeline:

1. **Entity Extraction:** Aus dem Aufgabentext werden genannte Funktionsnamen, Dateinamen und Schlüsselbegriffe extrahiert (via kleinem LLM-Call oder Regex-Matching).
2. **Seed-Node-Identifikation:** Die extrahierten Entitäten werden im Graphen als Startknoten lokalisiert.
3. **Graph-Traversal:** Von den Startknoten aus wird der Graph in konfigurierbarer Hop-Tiefe traversiert. Unterschiedliche Traversal-Strategien sind wählbar: Strukturell (Aufruf- und Importkanten), historisch (häufig co-modifizierte Knoten) und testbasiert (zugehörige Testknoten).
4. **Ranking und Filterung:** Knoten werden nach Relevanz geordnet (Hop-Distanz, Testabdeckung, Risikogewicht) und auf ein konfigurierbares Tokenbudget begrenzt.

5. **Serialisierung:** Der Teilgraph wird als strukturierter Kontext-Block (Markdown oder JSON) ausgegeben, inklusive expliziter Angaben, *warum* jeder Knoten aufgenommen wurde.

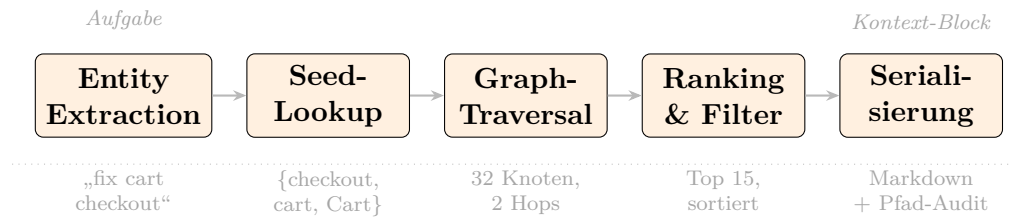


Abbildung 2: Fünfstufige Context-Assembly-Pipeline (get_context). Die untere Zeile zeigt ein Beispiel für die Aufgabe „fix cart checkout“.

Transparenz als First-Class-Feature

Jeder ausgelieferte Kontext-Block enthält einen *Entscheidungspfad*: Eine maschinenlesbare Begründung, über welche Kanten jeder Knoten erreicht wurde (checkout() aufgenommen via CALLS-Kante von cart.ts, Zeile 42). Entwickler können diesen Teilgraphen vor Agentausführung inspizieren und gezielt erweitern oder beschneiden. Nach Abschluss der Aufgabe dient der Entscheidungspfad als Audit-Trail.

Integration via MCP

Das System wird als *MCP-Server* implementiert und stellt drei Tools bereit:

get_context(task) Assembliert und gibt den Kontext-Block für eine Aufgabenbeschreibung zurück.

inspect_graph(node_id) Gibt den direkten Nachbarschaft-Subgraphen eines Knotens zurück.

update_graph() Triggert manuell eine Graph-Aktualisierung (außerhalb des automatischen Git-Hooks).

Claude Code, Cursor und andere MCP-kompatible Clients können diese Tools direkt aufrufen. Damit ist das System ohne Änderung am Agent selbst nutzbar.

Stand der Vorarbeiten

Proof-of-Concept bereits implementiert

Im Rahmen der Einarbeitungsphase wurde ein funktionsfähiger Proof-of-Concept für die TypeScript-Komponente entwickelt und auf dem eigenen WebshopTemplate-Projekt validiert:

- **Parser (M2):** tree-sitter-Parser für TypeScript/TSX extrahiert Datei-, Funktions- und Klassenknoten sowie IMPORTS-, CALLS-, EXTENDS- und DEFINED_IN-Kanten.
- **Graph (M3):** Neo4j-Instanz mit 59 Dateiknoten, 64 Funktionsknoten und 25 aufgelösten CALLS-Kanten; Git-History-Metriken (`commitCount`, `lastCommitMessage`) als Knotenattribute; Performance-Indexe.
- **Context Assembly (M5):** Fünfstufige Pipeline (`get_context`) mit Entity Extraction, Seed-Node-Identifikation, konfigurierbarer Graphtraversierung, Relevanz-Ranking und Markdown-Serialisierung inklusive Entscheidungspfad.
- **MCP-Server (M6):** Drei Tools (`get_context`, `inspect_graph`, `update_graph`) als MCP-Server implementiert und in Claude Code registriert.

Die Living-Eigenschaft (automatischer Git-Hook, M4) und die Python-Komponente sind in der eigentlichen Masterarbeit zu entwickeln.

5 Methodisches Vorgehen

Forschungsrahmen: Design Science Research

Die Arbeit folgt dem *Design Science Research* (DSR)-Paradigma nach Hevner u. a. (2004). DSR ist für informatiknahe Masterarbeiten geeignet, die ein nützliches Artefakt entwickeln und dessen Wirksamkeit empirisch nachweisen. Der Zyklus umfasst drei Phasen: *Relevance* (das Problem ist real und praxisrelevant), *Design* (das Artefakt wird iterativ entwickelt und verfeinert) und *Rigor* (die Evaluation ist methodisch fundiert). Für diese Arbeit bedeutet das: Das System wird nicht einmalig gebaut und dann evaluiert, sondern über mehrere Build-Evaluate-Zyklen iteriert.

Evaluation: Drei Versuchsbedingungen

Die Evaluation vergleicht drei Bedingungen:

B1: Reaktive Exploration Der Coding Agent erhält nur die Aufgabenbeschreibung und exploriert die Codebase selbständig (Baseline, entspricht dem aktuellen Stand der Praxis).

B2: Statischer Kontext Der Agent erhält zusätzlich eine statisch gepflegte `CLAUDE.md` mit Architekturhinweisen und Konventionen. Diese Bedingung entspricht dem heutigen Best-Practice und ist empirisch fundiert: Entwickler-geschriebene Kontext-Dateien verbessern die Lösungsrate um durchschnittlich 4 %, reduzieren aber die Inferenzeffizienz (Gloaguen u. a. 2026; Lulla u. a. 2026).

B3: Living Knowledge Graph Der Agent erhält den durch das System assemblierten, aufgabenspezifischen Kontext-Block inklusive Entscheidungspfad (das neue Artefakt).

Bedingung	T1: Bugfixes	T2: Refactorings	T3: Features
B1 Reaktive Exploration	Baseline	Baseline	Baseline
B2 Statisch (CLAUDE.md)	HF1	HF1	HF1
B3 Living Knowledge Graph	HF1 · HF2	HF1 · HF2	HF1 · HF2 · HF3

Metriken je Zelle: Token-Verbrauch, Iterationsanzahl, Erstkorrektheit, Kontextpräzision. HF3 (Retrieval-Transparenz) wird über alle B3-Zellen erhoben; T3 ist der primäre Messkontext.

Abbildung 3: Evaluationsdesign: drei Versuchsbedingungen (B1 bis B3) gekreuzt mit drei Aufgabentypen (T1 bis T3). Jede Zelle steht für einen Messdurchlauf.

Versuchsumgebungen

Die Evaluation wird auf zwei bis drei laufenden Hochschulprojekten des Betreuers durchgeführt, also aktiv weiterentwickelten Python- und TypeScript-Projekten mit gewachsener Codebasis. Diese Umgebungen bieten realistische, nicht konstruierte Aufgaben und ermöglichen die Beteiligung von Studierenden als Probanden. Während industrielle Studien wie Sherrier u. a. (2024) Coding Agents auf realen Projekten evaluieren und

dabei Durchsatz und Akzeptanzraten messen, steht bei dieser Arbeit die Fähigkeit der Entwicklenden im Vordergrund, die Kontextauswahl des Agenten zu verstehen und gezielt zu korrigieren, eine Dimension, die in bestehenden Feldevaluationen bislang kaum erfasst wird.

Pro Projekt werden Aufgaben dreier Typen bearbeitet:

- T1: Isolierte Bugfixes** Lokal begrenzte Korrekturen.
- T2: Multi-File-Refactorings** Strukturelle Änderungen mit dateiübergreifenden Auswirkungen.
- T3: Feature-Entwicklungen** Neue Funktionalität mit Architektur- und Konventionsbedarf.

Metriken

Dimension	Operationalisierung	Erhebung
Effizienz	Iterationsanzahl bis zur akzeptierten Lösung	Logging
Token-Verbrauch	Input- und Output-Tokens pro Aufgabe	API-Logs
Erstkorrektheit	Anteil Aufgaben, die im ersten Durchlauf alle Tests bestehen	Test-Suite
Kontextpräzision	Anteil relevanter zu insgesamt ausgelieferter Kontext-Tokens	Annotierung
Transparenz	Entwickler-Bewertung der Verständlichkeit des Entscheidungspfads	Kurzumfrage

Validität und Abgrenzung

Die interne Validität wird durch reproduzierbare Aufgaben, fixierte Modellversionen und mehrfache Wiederholungen je Bedingung gesichert. Die externe Validität ergibt sich aus der Nutzung realer, aktiv betreuter Hochschulprojekte statt konstruierter Benchmarks. Eine Einschränkung ist die Beschränkung auf die Sprachen Python, JavaScript und TypeScript sowie auf MCP-kompatible Coding Agents.

6 Vorläufige Gliederung der Arbeit

- 1. Einleitung** Motivation, Problemstellung, Zielsetzung und Aufbau der Arbeit.

- 2. Grundlagen** Große Sprachmodelle für Code; agentische Architekturen (ReAct, Werkzeugnutzung); Wissensgraphen und Graphdatenbanken; Kontext- und Prompt-Engineering; Model Context Protocol.
 - 3. Stand der Forschung** Agentische Ansätze für Software Engineering (SWE-agent, Agentless); graphbasierte Kontextrepräsentation (RepoGraph, KGCompass); proaktive vs. reaktive Kontextbereitstellung (CodeScout, ContextBench); statische Kontext-Dateien als Agenten-Baseline (AGENTS.md-Literatur); Ableitung der Forschungslücke.
 - 4. Systemarchitektur** Detaillierte Beschreibung der drei Schichten: Ingestion (tree-sitter, Parser-Design), Living Graph (Neo4j-Schema, Git-Synchronisation) und Context Assembly (Traversal-Strategien, Ranking, Serialisierung); MCP-Integration; Transparenzkonzept.
 - 5. Implementierung** Technische Umsetzung in Python und TypeScript; Deployment auf den Hochschulprojekten; Logging-Pipeline; Aufgabenkatalog.
 - 6. Ergebnisse** Quantitative Befunde (B1 vs. B2 vs. B3 nach Aufgabentyp und Metrik); qualitative Beobachtungen aus Agent-Logs; Transparenz-Feedback der Entwickler.
 - 7. Diskussion** Interpretation der Ergebnisse; Rückbezug auf Hypothesen; Traversal-Strategien im Vergleich; Grenzen des Ansatzes; praktische Implikationen für Kontext-Engineering.
 - 8. Fazit und Ausblick** Zusammenfassung der Beiträge; offene Forschungsfragen; mögliche Erweiterungen (weitere Sprachen, automatische Traversal-Parametrisierung, LLM-gestützte Graphanreicherung).
- Anhang** Aufgabenkatalog; Graph-Schemata; Prompt-Templates; auszugsweise Roh-Logs; ergänzende Tabellen.

7 Zeitplan

Die Bearbeitung ist auf 24 Wochen angelegt. Die ersten sechs Wochen sind Grundlagen und Systemkern gewidmet, sodass ab Woche 7 ein laufendes System existiert, das iterativ auf den Hochschulprojekten verfeinert wird.

Wochen	Phase / Aktivität	Meilenstein
W01–W02	Literaturvertiefung; Absprache Hochschulprojekte; DSR-Design finalisieren	M1: Design final
W03–W04	tree-sitter Parser für Python + TypeScript/JavaScript; Neo4j-Schema	M2: Parser läuft*
W05–W06	Graph-Builder (Ingestion Layer); Erstbefüllung eines Hochschulprojekts	M3: Graph befüllt*
W07–W08	Git post-commit Hook; inkrementelle Synchronisation (Living Layer)	M4: Living Graph aktiv
W09–W10	Context Assembly: Traversal-Strategien, Ranking, Serialisierung	M5: Assembly läuft*
W11–W12	MCP-Server; Integration in Claude Code; erster End-to-End-Test	M6: MCP integriert*
W13–W15	Transparenz-Features: Entscheidungspfade, Subgraph-Visualisierung	M7: Transparenz fertig
W16–W18	Evaluation auf allen Hochschulprojekten; Datenerhebung B1 vs. B2 vs. B3	M8: Daten erhoben
W19–W21	Datenanalyse (quantitativ + qualitativ); Iteration am System	M9: Ergebnisse konsolidiert
W22–W24	Verschriftlichung; Review; Korrekturen; Abgabe	M10: Abgabe

* Im Rahmen der Einarbeitungsphase für TypeScript/WebshopTemplate bereits abgeschlossen (siehe Abschnitt 4).

Risikobetrachtung

R1: Parser-Abdeckungslücken tree-sitter-Grammatiken decken nicht alle Python- und TypeScript-Konstrukte vollständig ab. *Gegenmaßnahme:* Frühzeitiger Test auf den Ziel-Projekten in W03–W04; Fallback auf vereinfachtes Kanten-Set.

R2: Zugang zu Hochschulprojekten Nicht alle Projekte sind sofort zugänglich. *Gegenmaßnahme:* Frühzeitige Abstimmung in W01; paralleles Arbeiten mit dem eigenen WebshopTemplate als Ersatz-Lab.

R3: Neo4j-Performance bei großen Graphen Bei Projekten mit mehr als 50 000 Knoten können Traversal-Anfragen langsam werden. *Gegenmaßnahme:* Indexierung kritischer Felder ab W06; Subgraph-Caching für häufige Queries.

R4: Modell-Updates Anbieterseitige Änderungen an Claude Code oder der API können

Evaluationsergebnisse beeinflussen. *Gegenmaßnahme:* Modellversion und API-Version für die gesamte Erhebungsphase fixieren und dokumentieren.

R5: Token- und API-Kosten Wiederholte agentische Läufe für B1–B3 können das Budget überschreiten. *Gegenmaßnahme:* Vorabkalkulation pro Bedingung; Priorisierung der aussagekräftigsten Aufgaben-Typ-Kombinationen.

8 Einarbeitungshinweise

Dieser Abschnitt dokumentiert die Einarbeitungsphase (W01–W02), deren praktische Ergebnisse als Proof-of-Concept in Abschnitt 4 beschrieben sind. Die Ressourcen sind nach Priorität geordnet.

1. tree-sitter: Code-Parser

tree-sitter ist der technische Kern der Ingestion-Schicht. Priorität: **hoch**.

Einstieg Offizielle Dokumentation: <https://tree-sitter.github.io/tree-sitter/using-parsers>. Den Abschnitt „Using Parsers“ und „The Node API“ vollständig lesen.

Python-Bindings Paket `tree-sitter` + `tree-sitter-python` und `tree-sitter-typescript` installieren. Ein kleines Skript schreiben, das eine Python-Datei parst und alle Funktionsnamen + Signaturen ausgibt; das ist der Kern-Use-Case für diese Arbeit.

Übung Aus einem echten Python-Projekt (z. B. dem WebshopTemplate) alle `import`-Statements extrahieren und als Kantenliste ausgeben: `datei_a.py` `IMPORTS` `symbol_x` `FROM` `datei_b.py`.

Wichtig zu verstehen Der Unterschied zwischen einem *Syntax-Tree* (was tree-sitter liefert) und einem *Scope-Tree* (was für Aufrufkanten nötig ist). Für einfache Aufrufkanten reicht tree-sitter; für präzises Cross-File-Calling ist zusätzliche Auflösung nötig.

2. Neo4j: Graphdatenbank

Neo4j ist die Persistenzschicht. Priorität: **hoch**.

Kurs Neo4j GraphAcademy (kostenlos): <https://graphacademy.neo4j.com>. Dort die Kurse „Neo4j Fundamentals“ und „Cypher Fundamentals“ absolvieren. Zusammen ca. 4 Stunden.

Lokaler Start Neo4j Desktop herunterladen oder AuraDB Free Tier (Cloud, kein Setup): <https://neo4j.com/cloud/platform/aura-graph-database/>. AuraDB empfiehlt sich für den Einstieg.

Python-Driver neo4j Python-Paket. Eine kleine Pipeline schreiben, die den tree-sitter-Output in Neo4j-Knoten und -Kanten überführt.

- Wichtige Cypher-Abfragen üben:**
- Alle Funktionen die eine bestimmte Funktion aufrufen (eingehende CALLS-Kanten)
 - n-Hop-Nachbarschaft eines Knotens (`MATCH path = (n)-[*1..3]->(m)`)
 - Kürzester Pfad zwischen zwei Knoten

3. Pflichtlektüre: Kernpaper

Diese Paper sollten vor dem Beginn der Implementierung gelesen werden, in dieser Reihenfolge:

1. **KGCompass** (B. Yang u. a. 2025): Direkt verwandter Ansatz; Graph-Schema und Multi-Hop-Traversal genau analysieren. Besonders: Abschnitte zu Graph-Konstruktion und Fault-Localization.
2. **RepoGraph** (Ouyang u. a. 2025): Zeigt, wie ein Code-Graph als Plug-in-Modul funktioniert; Ego-Graph-Konzept ist relevant für die Context-Assembly-Schicht.
3. **SWE-agent** (J. Yang u. a. 2024): Zeigt wie reaktive Agent-Computer-Interfaces funktionieren; Grundlage für Bedingung B1 in der Evaluation.
4. **Agentless** (Xia u. a. 2024): Zeigt, dass prozedurale Kontextvorbereitung reaktive Exploration schlagen kann; wichtig für die Argumentation in HF1.
5. **ContextBench** (Li u. a. 2026): Empirische Grundlage für das Over-Retrieval-Problem; Benchmark-Design studieren für eigene Evaluation.

4. Model Context Protocol (MCP)

MCP ist die Integrationsschicht. Priorität: **mittel** (nach W10).

Dokumentation <https://modelcontextprotocol.io>: Abschnitte „*Introduction*“, „*Architecture*“ und „*Building Servers*“ lesen.

Python-SDK <https://github.com/modelcontextprotocol/python-sdk>: Das offizielle Tutorial „*Building a simple MCP server*“ vollständig durcharbeiten. Ein MCP-Server der ein einziges Tool (`hello_world`) bereitstellt, in unter einer Stunde gebaut.

Integration in Claude Code In Claude Code: `/mcp` Kommando zum Registrieren eigener Server. Der eigene MCP-Server kann damit direkt in der Entwicklungsumgebung getestet werden, ohne eine separate App.

5. Design Science Research

Für die methodische Einbettung der Arbeit. Priorität: **mittel**.

Grundlagentext Hevner u. a. (2004): „*Design Science in Information Systems Research*“, MIS Quarterly 28(1), 2004. Besonders die drei Design-Science-Zyklen (*Relevance Cycle*, *Design Cycle*, *Rigor Cycle*) internalisieren; das ist das Gerüst für Kapitel 5 der Arbeit.

Praxisorientierter Einstieg Peffers, K. et al. (2007): „*A Design Science Research Methodology for Information Systems Research*“, JMIS 24(3). Gibt eine konkrete sechsstufige Prozessstruktur, die sich direkt auf diese Arbeit mappen lässt.

6. Empfohlene GitHub-Repositories zum Erkunden

tree-sitter/tree-sitter <https://github.com/tree-sitter/tree-sitter>: Hauptrepository; Grammar-Dateien für Python und TypeScript.

modelcontextprotocol/python-sdk <https://github.com/modelcontextprotocol/python-sdk>: MCP Python-SDK mit Beispielen.

SWE-agent <https://github.com/SWE-agent/SWE-agent>: Referenzimplementierung reaktiver Agent-Computer-Interfaces; für das Verständnis von B1.

Agentless <https://github.com/OpenAutoCoder/Agentless>: Referenzimplementierung prozeduraler Kontextzusammenstellung; für das Verständnis des proaktiven Ansatzes.

7. Abgeschlossene erste praktische Schritte

1. tree-sitter installieren und das WebshopTemplate parsen: Alle TypeScript-Funktionen und ihre Imports als JSON ausgeben.
2. Diesen Output in eine Neo4j-AuraDB-Instanz schreiben: Knoten für Dateien und Funktionen, Kanten für IMPORTS.
3. Eine Cypher-Abfrage schreiben: „*Welche Funktionen werden von `CartContext.tsx` aufgerufen?*“ Das ist der Proof-of-Concept für den Kern des Systems.
4. Einen minimalen MCP-Server bauen, der diese Cypher-Abfrage als Tool bereitstellt, und ihn in Claude Code registrieren.

Diese vier Schritte wurden im Rahmen der Einarbeitungsphase abgeschlossen und belegen, dass die technischen Kernkomponenten zusammenspielen. Sie bilden die Grundlage des in Abschnitt 4 beschriebenen Proof-of-Concept.

Literaturverzeichnis

- Anthropic (2024). *Model Context Protocol: An Open Standard for LLM Tool Integration*. Veröffentlicht November 2024. URL: <https://modelcontextprotocol.io>.
- Gloaguen, Thibaud, Niels Mündler, Mark Müller, Veselin Raychev und Martin Vechev (2026). *Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?* arXiv preprint arXiv:2602.11988. URL: <https://arxiv.org/abs/2602.11988>.
- Hevner, Alan R., Salvatore T. March, Jinsoo Park und Sudha Ram (2004). „Design Science in Information Systems Research“. In: *MIS Quarterly* 28.1, S. 75–105. DOI: [10.2307/25148625](https://doi.org/10.2307/25148625).
- Hutter, Robert und Michael Pradel (2026). *AgentStepper: Interactive Debugging of Software Development Agents*. arXiv preprint arXiv:2602.06593. URL: <https://arxiv.org/abs/2602.06593>.
- Jimenez, Carlos E. u. a. (2024). „SWE-bench: Can Language Models Resolve Real-World GitHub Issues?“ In: *The Twelfth International Conference on Learning Representations (ICLR 2024)*. arXiv preprint arXiv:2310.06770. URL: <https://arxiv.org/abs/2310.06770>.
- Li, Han u. a. (2026). *ContextBench: A Benchmark for Context Retrieval in Coding Agents*. arXiv preprint arXiv:2602.05892. URL: <https://arxiv.org/abs/2602.05892>.
- Liu, Nelson F. u. a. (2024). „Lost in the Middle: How Language Models Use Long Contexts“. In: *Transactions of the Association for Computational Linguistics* 12, S. 157–173. DOI: [10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638). URL: <https://arxiv.org/abs/2307.03172>.
- Lulla, Jai Lal, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes und Christoph Treude (2026). *On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents*. arXiv preprint arXiv:2601.20404. URL: <https://arxiv.org/abs/2601.20404>.
- Ouyang, Siru u. a. (2025). „RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph“. In: *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. arXiv preprint arXiv:2410.14684. URL: <https://arxiv.org/abs/2410.14684>.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon und Mert Demirer (2023). *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. arXiv preprint arXiv:2302.06590. URL: <https://arxiv.org/abs/2302.06590>.
- Sherrier, David, Chun-Nan Chou, Lukas Voelcker, Piotr Mardziel, Rongzhen Gu u. a. (2024). *HULA: Human-in-the-Loop LLM-Based Agents for Software Development*. arXiv preprint arXiv:2411.12924. URL: <https://arxiv.org/abs/2411.12924>.

- Suri, Manan u. a. (2026). *CodeScout: Contextual Problem Statement Enhancement for Software Agents*. arXiv preprint arXiv:2603.05744. URL: <https://arxiv.org/abs/2603.05744>.
- Xia, Chunqiu Steven, Yinlin Deng, Soren Dunn und Lingming Zhang (2024). *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv preprint arXiv:2407.01489. URL: <https://arxiv.org/abs/2407.01489>.
- Yang, Boyang u. a. (2025). *KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair*. arXiv preprint arXiv:2503.21710. URL: <https://arxiv.org/abs/2503.21710>.
- Yang, John u. a. (2024). „SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering“. In: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. arXiv preprint arXiv:2405.15793. URL: <https://arxiv.org/abs/2405.15793>.